

Rapid terrain adaptation through continual learning of central pattern generator parameters for quadrupedal robot locomotion

Alex Pitoiset¹, Tim N. Nisslbeck¹, Ajith Anil Meera¹ and Wouter M. Kouw¹

Abstract—

I. INTRODUCTION

Achieving the same level of adaptability as animals is crucial for deploying legged machines in constantly changing real world environments. To bridge the gap between engineering and nature, engineers have frequently turned to biology for ideas, replicating animal morphologies. For example, snake-like robots for constrained environments or quadruped platforms for irregular terrain [?], [?]. These morphologies inherit the mechanical advantages of their biological counterparts. However, their control systems often lack in adaptability. For example, the Boston Dynamics quadrupedal robot, Spot, mimics animal legs. However, it relies on centralized and computationally intensive controllers that require multiple sensors [?]. Although this approach performs well, it remains complex and expensive, limiting its scalability for wider deployment and access.

To address these challenges, a promising approach to improve robotic locomotion is to use Central Pattern Generators (CPGs), which are neural circuits that generate high-dimensional rhythmic signals. In vertebrates, CPGs are located in the spinal cord and produce periodic coordinated movements such as walking and breathing [?]. By automating these repetitive physical patterns, CPGs allow the brain to redirect its resources to other cognitive tasks, such as selecting safe paths or responding to unexpected stimuli. These systems are triggered by simple inputs from higher brain centers. The brain can modulate their rhythmic output to control parameters such as locomotion speed and gait. Moreover, CPGs remain coupled to the body and environment through sensory feedback [?]. This decentralized architecture reduces the delays for commands and the required bandwidth while simplifying high-level control [?]. These features make CPG-based controllers an attractive solution as they are less energy intensive, simpler, and more affordable.

A. Related Work

Inspired by biological principles, various CPG models have been implemented in robotics. Among them, coupled nonlinear oscillators have emerged as biologically plausible and computationally efficient for generating rhythmic movement [?], [?].

The power of coupled nonlinear oscillators lies in their ability to inhibit each other. This coupling behavior enables synchronization between joints, producing coordinated gaits and smooth transitions between them [?], [?], [?]. Such mechanisms mimic biological inter-neuronal inhibition and allow the generation of out-of-phase rhythmic signals. They also exhibit a limit-cycle behavior, producing periodic and self-sustaining oscillations that quickly recover from transient perturbations [?]. In addition, their compact dynamics, governed by a small set of parameters (e.g. frequency and amplitude), makes them appropriate for embedded systems with limited resources. Despite their promise, most existing implementations target high-end robotic platforms. Their applicability to compact, affordable systems remains under-explored, which underpins the present study.

In the following, we contribute to the field with

- the design of an intelligent agent that continuously updates parameters of a central pattern generator, and
- by benchmarking this agent against offline grid search and online Bayesian optimization.

II. PROBLEM STATEMENT

The Peto Bittle quadruped robot should execute a stable forward walk without falling. The robot has four legs, each with two joints controlled by servomotors: a hip joint $v_i(t)$ and a knee joint $w_i(t)$ for leg $i = 1, \dots, 4$. At each discrete time step k , with fixed sampling interval Δt , the pose of the robot's base is described by its linear position $\mathbf{q}_k = (q_k^x, q_k^y, q_k^z)$ and its orientation in Euler angles $\gamma_k = (\gamma_k^p, \gamma_k^r, \gamma_k^y)$, where q_k^x is the forward displacement along the x -axis, q_k^y and q_k^z represent lateral and vertical displacements, and γ_k^p , γ_k^r , and γ_k^y denote pitch, roll, and yaw angles, respectively. Joint trajectories $v_i(t)$, $w_i(t)$ are subject to two performance criteria. First, forward progression must meet a minimum target distance D_{target} over a time $T = \frac{N}{\Delta t}$, where N is the number of time steps, such that $q_T^x \geq D_{\text{target}}$. Simultaneously, the robot must maintain stability, which means that lateral and vertical deviations, as well as orientation errors, remain negligible: $|q_k^y| \approx 0$, $|q_k^z - H| \approx 0$, and $|\gamma_k^p| \approx 0$, $|\gamma_k^r| \approx 0$, $|\gamma_k^y| \approx 0$, where H is the nominal body height.

Second, the CPG must exhibit stable limit-cycle behavior so that transient perturbations are rapidly attenuated and do not lead to falls. A fall is declared if $q_k^z < h_{\min}$ or $|\gamma_k^r| > \gamma_{\max}$ at any time step, where $h_{\min}$ and $\gamma_{\max}$ are safety thresholds.

From a control perspective, the Hopf oscillator network should allow independent modulation of the descending (stance) and ascending (swing) phases, reflecting the asymmetric nature of animal locomotion [?]. In fact, in stable

*This work was supported by the Eindhoven Artificial Intelligence Systems Institute

¹Department of Electrical Engineering, Eindhoven University of Technology, Netherlands w.m.kouw@tue.nl

quadrupedal gaits, the stance phase typically occupies a longer duration than the swing phase. It should also allow smooth adjustment of amplitude and frequency via simple parameter changes and strong coupling with the robot's mechanics. The overall architecture must remain generic enough to accommodate different quadruped robots and to allow future integration of higher-level command signals.

III. METHODS

A. Central Pattern Generator

The CPGs are modeled as a network of four mutually coupled Hopf oscillators, each driving one limb: left front (LF), right front (RF) left hind (LH) and right hind (RH). The coordination between limbs arises from inhibitory coupling. For instance, for the trot gait, oscillator 1 (LF) is excited by 4 (RH) $k_{14} = +1$, while 2 (RF) and 3 (LH) inhibit 1 (LF), as $k_{12} = -1$ and $k_{13} = -1$.

Each oscillator is governed by a modified Hopf model. It allows phase-dependent frequency modulation and enables independent control over stance and swing durations. The state of each oscillator (x_i, y_i) depends on the following set of equations:

$$\dot{x}_i = \alpha(u - r_i^2)x_i - \omega_i y_i, \quad (1)$$

$$\dot{y}_i = \beta(u - r_i^2)y_i + \omega_i x_i + \gamma \sum_{j=1}^4 k_{ij} y_j, \quad (2)$$

$$\omega_i = \frac{\omega_{\text{stance}}}{1 + e^{-by_i}} + \frac{\omega_{\text{swing}}}{1 + e^{by_i}}, \quad (3)$$

where $r_i = \sqrt{x_i^2 + y_i^2}$ is the instantaneous amplitude, u defines the squared amplitude of the limit cycle, and $\alpha, \beta > 0$ are convergence rates (typically $\beta \gg \alpha$ to enforce stronger convergence along the y -axis). The term ω_i transitions between the stance and swing frequencies depending on the phase y_i , and γ is the coupling gain added to have stronger coupling and less dependency on the initial conditions settings. The sigmoid functions in Equation 3 ensure smooth and continuous transition between the stance and swing frequencies. A sigmoid is an S-shaped mathematical function that smoothly interpolates between two values; in this context, it gradually shifts the oscillator's frequency based on the sign and magnitude of y_i . The parameter b sets the sharpness between this transition. This allows the oscillator to gradually change between the swing and the stance phases according to the sign and magnitude of y_i . The coupling matrix $K = [k_{ij}]$ is gait-specific.

Figure ?? shows the time evolution of the outputs from the Hopf CPG model and the joint angles for the walking gait simulation. The CPGs demonstrate the expected sinusoidal movement for each leg. Figure ?? shows the limit cycle behavior of the CPG, where the state variables x_i and y_i converge to a circular trajectory of radius $\sqrt{\mu}$ and closely follow this limit cycle over time. This demonstrates the oscillator's stability and periodicity, which are critical for generating smooth, consistent joint trajectories necessary for coordinated legged locomotion.

B. Mapping to Joint Angles

The output from the Hopf CPG model, represented by the variables x_i and y_i , is mapped to the robot's joint angles, determining how the simulated CPG behavior translates into actual limb movement. Specifically, the hip and knee joint angles for each leg are computed as follows:

$$\theta_{\text{hip}} = \theta_{\text{hip base}} + \theta_{\text{hip range}} \cdot x_i \quad (4)$$

$$\theta_{\text{knee}} = \theta_{\text{knee base}} - \theta_{\text{knee range}} \cdot \max(0, y_i) \quad (5)$$

Here, x_i modulates the hip motion, corresponding to the forward-backward swing of the leg, while y_i controls knee flexion during the swing phase. Negative values of y_i are suppressed to prevent unnatural hyperextension. This mapping transforms the original circular limit cycle into a biologically inspired trajectory for the foot, more closely resembling the natural gait cycle observed in animals. It allows the robot to generate plausible leg trajectories that contribute to effective and natural locomotion.

C. Autoregressive Active Inference agent

The agent is based on a multivariate autoregressive model with exogenous inputs [?]

At the core of the model is a vector autoregressive process with a regression coefficient matrix $A \in \mathbb{R}^{D_x \times D_y}$ and a noise precision matrix $W \in \mathbb{R}_+^{D_y \times D_y}$ where $\mathbb{R}_+$ refers to the space of positive semi-definite matrices. Let $\Theta = (A, W)$ refer to the parameters jointly. The autoregressive model forms our likelihood function:

$$p(y_k | \Theta, u_k, \bar{u}_k, \bar{y}_k) = \mathcal{N}(y_k | A^T x_k, W^{-1}), \quad (6)$$

where $\bar{u}_k, \bar{y}_k$ are running memories of previous inputs and outputs, respectively, and x_k is the concatenation of the current control and the memories:

$$\bar{u}_k = \begin{bmatrix} u_{k-1} \\ \vdots \\ u_{k-M_u} \end{bmatrix}, \quad \bar{y}_k = \begin{bmatrix} y_{k-1} \\ \vdots \\ y_{k-M_y} \end{bmatrix}, \quad x_k = \begin{bmatrix} u_k \\ \bar{u}_k \\ \bar{y}_k \end{bmatrix}. \quad (7)$$

The vector x_k is of dimension $D_x = D_u(M_u + 1) + D_y M_y$. Note that although the memories are indexed with time k , they do not contain the current input u_k and output y_k , only the previous inputs and outputs.

The prior distribution on the parameters is a matrix normal Wishart distribution [?, ID: P323]:

$$p(\Theta) = p(A | W) p(W) \quad (8)$$

$$= \mathcal{M} \mathcal{N}(A | M_0, \Lambda_0^{-1}, W^{-1}) \mathcal{W}(W | \Omega_0^{-1}, \nu_0). \quad (9)$$

The matrix normal distribution for the coefficients is

$$\mathcal{M} \mathcal{N}(A | M_0, \Lambda_0^{-1}, W^{-1}) = \quad (10)$$

$$\sqrt{\frac{|W|^{D_x} |\Lambda_0|^{D_y}}{(2\pi)^{D_x D_y}}} \exp\left(-\frac{1}{2} \text{tr}[W(A - M_0)^T \Lambda_0 (A - M_0)]\right),$$

and the Wishart distribution for the noise precision matrix is

$$\mathcal{W}(W | \Omega_0^{-1}, \nu_0) = \quad (11)$$

$$\sqrt{\frac{|\Omega_0|^{\nu_0}}{2^{\nu_0 D_y}}} \frac{\sqrt{|W|^{\nu_0 - D_y - 1}}}{\Gamma_{D_y}(\nu_0/2)} \exp\left(-\frac{1}{2} \text{tr}[W \Omega_0]\right).$$

The prior distributions over control inputs are assumed to be independent over time:

$$p(u_k) = \mathcal{N}(u_k | 0, \Upsilon^{-1}), \quad (12)$$

with precision matrix Υ . This choice has a regularizing effect on the inferred controls (Sec. III-C.1.b). Finally, we have a probability distribution indicating the target observation for the system to reach;

$$p(y_t | y_*) = \mathcal{N}(y_t | m_*, S_*). \quad (13)$$

Note that y_* refers to the goal prior parameters, m_*, S_* . A goal prior is a richer distribution of a target than a numerical set-point: it allows one to indicate, for instance, that the agent's task is complete when it has reached an area of high probability mass.

1) *Inference*: Our inference procedure is separated into a parameter belief update procedure given observed data, and a control estimation procedure given planned trajectories.

a) *Parameters*: Given observed y_k and u_k , we update our parameter belief through Bayesian filtering [?]:

$$\underbrace{p(\Theta | \mathcal{D}_k)}_{\text{posterior}} = \underbrace{\frac{p(y_k | \Theta, u_k, \bar{u}_k, \bar{y}_k)}{p(y_k | u_k, \mathcal{D}_{k-1})}}_{\text{evidence}} \underbrace{p(\Theta | \mathcal{D}_{k-1})}_{\text{prior}}. \quad (14)$$

where $\mathcal{D}_k = \{y_i, u_i\}_{i=1}^k$ is short-hand for data up to time k . Note that the memories $\bar{u}_k, \bar{y}_k$ are subsets of $\mathcal{D}_{k-1}$. The evidence term is the evaluation of the observation y_k under the predictive distribution, obtained by marginalizing the likelihood over the parameters.

Marginalizing the Gaussian likelihood in Eq. 6) over a matrix normal Wishart distribution (Eq. 8), with parameters $M_{k-1}, \Lambda_{k-1}, \Omega_{k-1}$ and v_{k-1} , yields a multivariate location-scale T-distribution:

$$p(y_k | u_k, \mathcal{D}_{k-1}) = \mathcal{T}_{\eta_k}(y_k | \mu_k(u_k), \Sigma_k(u_k)), \quad (15)$$

whose parameters are $\eta_k = v_{k-1} - D_y + 1$,

$$\mu_k(u_k) = M_{k-1}^\top \begin{bmatrix} u_k \\ \bar{u}_k \\ \bar{y}_k \end{bmatrix} \quad (16)$$

$$\Sigma_k(u_k) = \frac{1}{v_{k-1} - D_y + 1} \Omega_{k-1} \left(1 + \begin{bmatrix} u_k \\ \bar{u}_k \\ \bar{y}_k \end{bmatrix}^\top \Lambda_{k-1}^{-1} \begin{bmatrix} u_k \\ \bar{u}_k \\ \bar{y}_k \end{bmatrix} \right). \quad (17)$$

The proof is in Appendix ?? . Note that the parameters of the predictive distribution are functions of the control u_k .

The posterior distribution over the parameters Θ (Eq. 8) is proportional to a matrix normal Wishart posterior distribution:

$$p(\Theta | \mathcal{D}_k) = \mathcal{MNW}(A, W | M_k, \Lambda_k^{-1}, \Omega_k^{-1}, v_k). \quad (18)$$

The parameters of this distribution are

$$v_k = v_{k-1} + 1, \quad (19)$$

$$\Lambda_k = \Lambda_{k-1} + x_k x_k^\top, \quad (20)$$

$$M_k = (\Lambda_{k-1} + x_k x_k^\top)^{-1} (\Lambda_{k-1} M_{k-1} + x_k y_k^\top), \quad (21)$$

$$\Omega_k = \Omega_{k-1} + y_k y_k^\top + M_{k-1}^\top \Lambda_{k-1} M_{k-1} - (\Lambda_{k-1} M_{k-1} + x_k y_k^\top)^\top (\Lambda_{k-1} + x_k x_k^\top)^{-1} (\Lambda_{k-1} M_{k-1} + x_k y_k^\top). \quad (22)$$

b) *Controls*: Reinforcement learning produces a policy that maximizes a reward function. But this is not the only way to learn from interactions with an environment. Actions are latent variables in a model and can be inferred, just like any other latent variable. We follow the perspective of active inference [?], [?], where we infer a variational posterior distribution over actions using an *expected* free energy functional [?]. We first describe a 1-step ahead control procedure, where all parts of the agent's memory x_k are observed. We then generalize the procedure to longer time horizons.

We start with building a generative model for the input and output at time $t = k + 1$:

$$p(y_t, \Theta, u_t | \mathcal{D}_k) = p(y_t | \Theta, u_t, \bar{u}_t, \bar{y}_t) p(\Theta | \mathcal{D}_k) p(u_t). \quad (23)$$

We want the agent to pursue a target, a specific future observation. To do so, we first isolate the marginal distribution for the future output,

$$p(y_t | \Theta, u_t, \bar{u}_t, \bar{y}_t) p(\Theta | \mathcal{D}_k) = p(\Theta | y_t, u_t, \mathcal{D}_k) p(y_t), \quad (24)$$

and then constrain the marginal to a specific parametric distribution (the *goal prior*), $p(y_t) \rightarrow p(y_t | y_*)$.

The distribution over parameters given the future output and input, is unknown to us. We use Bayes' rule in the reverse direction to relate it to known distributions:

$$p(\Theta | y_t, u_t, \mathcal{D}_k) = \frac{p(y_t | \Theta, u_t, \bar{u}_t, \bar{y}_t) p(\Theta | \mathcal{D}_k)}{p(y_t | u_t, \mathcal{D}_k)}. \quad (25)$$

The marginal likelihood in the denominator is actually the posterior predictive distribution, since it is the marginalization of the likelihood of a future observation.

$$\mathcal{F}_k[q] = \mathbb{E}_{q(y_t, \Theta, u_t)} \left[\ln \frac{q(y_t, \Theta, u_t)}{p(y_t, \Theta, u_t | y_*, \mathcal{D}_k)} \right]. \quad (26)$$

where the variational model is specified to be:

$$q(y_t, \Theta, u_t) = q(y_t | \Theta, u_t) q(\Theta) q(u_t). \quad (27)$$

The first two factors are constrained to be equal to factors of the generative model,

$$q(y_t | \Theta, u_t) = p(y_t | \Theta, u_t, \bar{u}_t, \bar{y}_t) \quad (28)$$

$$q(\Theta) = p(\Theta | \mathcal{D}_k). \quad (29)$$

Inferring the optimal control at time t refers to minimizing the free energy functional with respect to the variational distribution $q(u_t)$:

$$q^*(u_t) = \arg \min_{q \in \mathcal{Q}} \mathcal{F}_k[q]. \quad (30)$$

where $\mathcal{Q}$ represents the set of probability distributions with support over $\mathcal{U}$. The optimal variational posterior under the free energy functional defined in (30) is:

$$q^*(u_t) = \frac{p(u_t) \exp(-\mathcal{J}(u_t))}{\int p(u_t) \exp(-\mathcal{J}(u_t)) du_t}, \quad (31)$$

where $\mathcal{J}(u_t) = D_{\text{KL}}[p(y_t | u_t, \mathcal{D}_k) \| p(y_t | y_*)]$ is the Kullback-Leibler divergence between the posterior predictive and the goal prior distribution.

We refer to the terms in Eq. ?? that are a function of $\Sigma(u_t)$ as "information-seeking" parts of the objective, because they bias the posterior towards actions that reduce uncertainty on future output predictions. We refer to terms that are a function $\mu(u_t)$ as "goal-seeking" because they bias the posterior towards actions that lead to future system outputs close to m_* , with respect to the metric imposed by the goal prior covariance matrix S_* . This is illustrated later in Section ??.

The generative model for time steps $\tau = (k+1, \dots, k+T)$ is:

$$p(y_\tau, \Theta, u_\tau | \mathcal{D}_k) = p(\Theta | \mathcal{D}_k) \prod_{t \in \tau} p(y_t | \Theta, u_t, \bar{u}_t, \bar{y}_t) p(u_t). \quad (32)$$

To impose the constraint on the marginal future output distributions, we require the marginal likelihood with respect to the product of autoregressive likelihoods;

$$p(y_\tau | u_\tau, \mathcal{D}_k) = \int p(\Theta | \mathcal{D}_k) \prod_{t \in \tau} p(y_t | \Theta, u_t, \bar{u}_t, \bar{y}_t) d\Theta. \quad (33)$$

Marginalizing over the parameter posterior $p(\Theta | \mathcal{D}_k)$ is challenging. Instead, we adopt an approximation commonly used in frequentist inference procedures [?], [?]: at each time $t \in \tau$, we replace the elements of $\bar{y}_t$ with the maximum a posteriori point estimates under their posterior predictive distributions. We denote this as $\hat{y}_t = [\mu_{t-1}(u_{t-1}) \ \mu_{t-2}(u_{t-2}) \ \dots]$. This means that we approximate the joint posterior predictive as

$$p(y_\tau | u_\tau, \mathcal{D}_k) = \prod_{t \in \tau} p(y_t | u_t, \mathcal{D}_k), \quad (34)$$

where for $t \geq k+2$, each posterior predictive is:

$$p(y_t | u_t, \mathcal{D}_k) = \mathcal{T}(y_t | M_k^\top \begin{bmatrix} u_t \\ \bar{u}_t \\ \hat{y}_{t-1} \end{bmatrix}, \frac{1}{V_k} \Omega_k (1 + \begin{bmatrix} u_t \\ \bar{u}_t \\ \hat{y}_{t-1} \end{bmatrix}^\top \Lambda_k^{-1} \begin{bmatrix} u_t \\ \bar{u}_t \\ \hat{y}_{t-1} \end{bmatrix}, \eta_t). \quad (35)$$

To summarize, we recursively apply the 1-step ahead prediction to build a T -step ahead prediction and an approximate control posterior for the entire planning horizon:

$$q^*(u_\tau) \propto \prod_{t \in \tau} p(u_t) \exp(-\mathcal{J}(u_t)). \quad (36)$$

Note that the goal prior is the same for each $t \in \tau$.

It is important to quantify uncertainty over controls. However, when interacting with the environment, we can ultimately only execute *one* action. For this, we calculate

the maximum a posteriori (MAP). The MAP estimate can be expressed as a minimization over a negative logarithmic transformation of $q^*(u_t)$:

$$\hat{u}_\tau = \arg \min_{u_\tau \in \mathbb{U}^T} -\frac{1}{2} \ln |\Sigma_\tau(u_\tau)| + \frac{1}{2} \text{tr} [S_*^{-1} \frac{\eta_\tau}{\eta_\tau - 2} \Sigma_\tau(u_\tau)] + \frac{1}{2} (\mu_\tau(u_\tau) - m_*)^\top S_*^{-1} (\mu_\tau(u_\tau) - m_*) + u_\tau^\top \Upsilon u_\tau. \quad (37)$$

$\mathbb{U}^T = \mathbb{U} \times \mathbb{U} \times \dots$ refers to the space of affordable controls over a planning horizon of length T , where practical constraints such as torque limits can be incorporated.

The autoregression coefficient matrix Λ_k , of size $D_x \times D_x$, has to be inverted each time-step. It is possible to reparameterize the matrix normal distribution in terms of a covariance matrix instead, but then that matrix will have to be inverted during parameter updates (Eq. 19). For numerical stability purposes, it is recommended to keep track of a Cholesky factor.

IV. EXPERIMENTS

We evaluate three parameter-selection strategies for the Hopf-oscillator CPG controller on the Laikago quadruped in PyBullet: (i) an offline *grid search* over a small set of CPG parameters; (ii) online *Bayesian optimization* (BO) with a Gaussian-process surrogate and a UCB acquisition function; and (iii) the proposed *MARX-EFE* active inference agent, which continually updates CPG parameters by minimizing expected free energy. Grid search serves as a fixed, non-adaptive reference point, BO serves as a state-of-the-art black-box optimizer, and MARX-EFE is the proposed continual learning agent.

All three methods optimize in the same 8-dimensional parameter space with identical bounds:

$$\theta = [\gamma, \omega_{\text{swing}}, \omega_{\text{stance}}, F_{\text{fast}}, K_{\text{stop}}, A_{\text{hip}}, A_{\text{knee}}, b],$$

corresponding to the coupling gain, the two phase-dependent frequencies, the Righetti-style fast-contact gain, the stop-gain, the hip and knee amplitudes, and the sigmoid sharpness b from Eq. 3. Each trial is a $T = 4.5$ s episode of simulated walking on flat ground with $\Delta t = 0.01$ s, a 1.5 s settling window excluded from the metrics, and a target forward velocity of 1.0 m/s. Parameter updates occur *online*: every trial carries the CPG state forward and smoothly interpolates between the previous and new parameters over the first 150 steps.

a) Performance metrics: We score each method on four quantities computed over the 3.0 s steady-state window: the scalar objective

$$J = w_1 R_v - w_2 \text{CoT} - w_3 \mathcal{S}, \quad (38)$$

with velocity reward $R_v = \frac{1}{T} \sum_k \exp(-\|v_k - v^*\|^2 / \sigma_v^2)$, cost of transport $\text{CoT} = \int P_{\text{mech}} dt / (mgd)$, and combined RMS stability $\mathcal{S} = \sqrt{\text{RMS}(\gamma^r)^2 + \text{RMS}(\gamma^p)^2}$; the forward distance d_y ; the lateral drift $|d_x|$; and the fall rate. To compare sample efficiency, we report the number of trials N_{walk} required to reach $J \geq J_{\text{th}}$ with cumulative fall rate below r_{th} , and the cumulative forward distance D_{cum} accumulated over the optimization horizon.

V. DISCUSSION

VI. CONCLUSIONS

APPENDIX

ACKNOWLEDGMENT